

Capítulo 0 - Docker

...

Introducción a docker



EL PROBLEMA DE LOS ENTORNOS DE EJECUCIÓN

El mismo código, diferentes resultados

Configurar
lleva tiempo,
que funcione en cualquier sitio
también es posible

En mi ordenador funciona...

"En mi ordenador funciona, pero en el tuyo no".

"Funciona en desarrollo, pero falla en pruebas".

"En producción da errores que no entiendo".



Una pequeña diferencia en el entorno puede provocar grandes problemas.

¿Pero si no he cambiado nada de código...?

Dependencias de una aplicación

Una aplicación necesita muchos componentes para funcionar. Todos ellos deben estar instalados y configurados correctamente.



Cuantas más dependencias, más posibilidades de que algo falle.

Versiones: un detalle importante

No solo importa qué componentes necesitas, sino también la versión de cada uno de ellos.



Java

- JDK 8
- JDK 11
- JDK 17
- JDK 21



Node.js

- v16
- v18
- v20
- v22



Bases de datos

- MySQL
- PostgreSQL
- Oracle
- SQL Server
- (y sus versiones)



Servidores

- Tomcat 9
- Tomcat 10
- WildFly
- Jetty
- ...



Una versión diferente puede hacer que la aplicación no funcione o se comporte de forma distinta.

Diferentes entornos: desarrollo, pruebas y producción

Una aplicación suele pasar por varios entornos, cada uno con sus propias características.

Desarrollo



- En el ordenador del desarrollador
- Cambios frecuentes
- Datos de prueba
- Versiones que pueden variar
- Más libertad para experimentar

Aquí se escribe y se prueba el código.

Pruebas (staging)



- Entorno más parecido a producción
- Datos de pruebas realistas
- Se valida que todo funciona
- Puede tener diferencias de configuración

Aquí se detectan problemas antes de llegar a producción.

Producción



- Entorno real, accesible por usuarios
- Datos reales
- Alta disponibilidad y seguridad
- Cambios controlados
- No se puede fallar

Aquí la aplicación debe funcionar siempre y para todos.

¿Qué es un entorno de ejecución?

El entorno de ejecución es el conjunto de software, librerías, herramientas y configuraciones necesarias para que una aplicación pueda ejecutarse correctamente.

Código de la aplicación



Entorno de ejecución

Sistema operativo

Runtimes (Java, Node, ...)

Librerías y dependencias

Configuración (variables, ficheros, ...)

Servicios (base de datos, servidor, ...)



Aplicación en funcionamiento



Si el entorno de ejecución cambia, el comportamiento de la aplicación también puede cambiar.



El reto: conseguir que nuestra aplicación se ejecute igual en cualquier entorno (desarrollo, pruebas, producción), de forma sencilla, reproducible y sin sorpresas.

Mismo código,
mismo comportamiento,
en cualquier lugar.



LA IDEA DE EMPAQUETAR UNA APLICACIÓN

Todo lo necesario, en un único paquete, listo para ejecutar

Construye hoy,
ejecuta en cualquier lugar

¿Qué significa “empaquetar”?

Consiste en reunir en un único paquete todo lo que necesita una aplicación para funcionar:

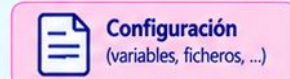
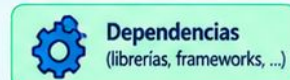
- El código de la aplicación
- Sus dependencias
- Su configuración
- Y todo lo necesario para ejecutarla



Así nos aseguramos de que siempre se ejecute igual, en cualquier entorno.

De muchos elementos a un único paquete

Una aplicación depende de muchos componentes. Los empaquetamos juntos para crear un entorno reproducible.



Paquete completo
(imagen de contenedor)

Aplicación + dependencias + configuración → entorno reproducible.

Una buena analogía: el contenedor de envío

Un contenedor de mercancías transporta todo lo necesario para que los productos lleguen en perfecto estado a cualquier lugar del mundo.



Con las aplicaciones pasa lo mismo: las empaquetamos para que funcionen igual en cualquier lugar.

Portabilidad

El paquete puede ejecutarse en diferentes entornos (desarrollo, pruebas, producción), sin necesidad de realizar cambios.



Construye una vez,
Ejecuta en cualquier lugar.

Reproducibilidad

El mismo paquete siempre se ejecuta de la misma forma, independientemente de dónde se despliegue.



Elimina el clásico
“en mi ordenador funciona”.

Aislamiento

Cada aplicación se ejecuta en su propio entorno, separado de las demás. Así se evitan conflictos entre versiones y dependencias.



Cada aplicación en su propio espacio.
Sin interferencias.

Automatización del despliegue

El paquete puede crearse, probarse y desplegarse automáticamente, facilitando procesos rápidos y fiables.



Menos tareas manuales,
más velocidad y menos errores.



¡Todo a bordo!
Tu aplicación, siempre lista
para la próxima aventura.

Empaquetar una aplicación nos permite construir entornos reproducibles, portátiles, aislados y listos para desplegar de forma automática.

Empaqueta hoy.
Despliega sin límites.



¿QUÉ ES DOCKER?

Construye, empaqueta y ejecuta tus aplicaciones en cualquier lugar

Misma aplicación,
mismos contenedores,
en cualquier entorno

Definición

Docker es una plataforma de contenedores que nos permite crear, empaquetar, distribuir y ejecutar aplicaciones de forma consistente en cualquier entorno.

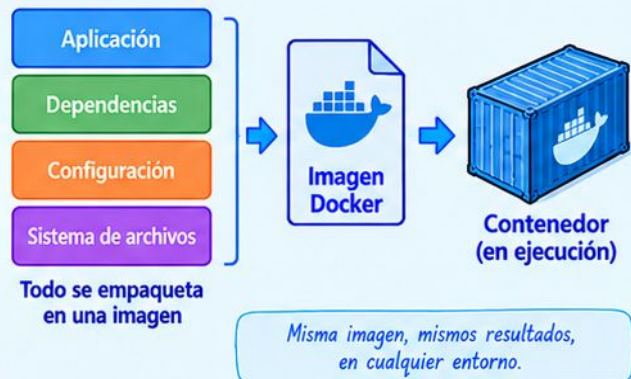
Docker se encarga de que tu aplicación funcione igual en tu ordenador, en pruebas y en producción.



Desarrolla hoy, despliega en cualquier lugar.

¿Cómo funciona?

Docker utiliza contenedores, que son entornos de ejecución aislados, ligeros y portátiles. A partir de una **imagen**, crea y ejecuta contenedores que incluyen todo lo necesario para que la aplicación funcione.



Componentes principales

Docker está formado por varios elementos que trabajan juntos:

Imágenes



- Plantillas de solo lectura
- Incluyen aplicación, dependencias y configuración
- Se usan para crear contenedores

Contenedores



- Instancias en ejecución de una imagen
- Aislados del sistema y de otros contenedores
- Ligeros y de arranque rápido

Docker Engine



- Runtime de Docker
- Se encarga de crear y gestionar imágenes y contenedores
- Se ejecuta como un servicio en segundo plano (daemon)

Docker CLI



- Interfaz de línea de comandos
- Permite interactuar con Docker Engine
- Ejemplos: docker build, docker run, docker ps, docker logs...

Docker Desktop

Docker Desktop es la aplicación oficial que facilita el uso de Docker en Windows, macOS y Linux. Incluye:

- Docker Engine
- Docker CLI
- Interfaz gráfica (Docker Desktop UI)
- Herramientas adicionales (Docker Compose, Kubernetes, etc.)



Windows



macOS



Linux

Instala Docker Desktop y empieza a trabajar con Docker de forma sencilla.

Ejemplo práctico

Con un solo comando podemos ejecutar un contenedor con una aplicación real:

```
docker run -d -p 8080:80 nginx
```

Este comando descarga la imagen de **nginx** (si no la tenemos), crea un contenedor y lo ejecuta, publicando el puerto 8080 de nuestro ordenador.

En pocos segundos, tienes un servidor web en funcionamiento.

Ventajas de Docker

- ✓ **Portabilidad:** funciona igual en cualquier entorno.
- ✓ **Reproducibilidad:** mismas imágenes, mismos resultados.
- ✓ **Aislamiento:** cada contenedor tiene su propio entorno.
- ✓ **Rapidez:** contenedores ligeros y de arranque rápido.
- ✓ **Facilidad de uso:** herramientas sencillas y automatización.
- ✓ **Gran ecosistema:** miles de imágenes disponibles en Docker Hub.

Docker Hub

Docker Hub es un registro público de imágenes Docker, con miles de imágenes oficiales y de la comunidad.



- Imágenes oficiales (nginx, mysql, node, ...)
- Imágenes de la comunidad
- Puedes publicar tus propias imágenes

Reutiliza, comparte y aprende de la comunidad.



En resumen: Docker es una plataforma de contenedores que te permite ejecutar aplicaciones de forma sencilla, consistente y portátil, con todas sus dependencias y configuración.

*Misma aplicación.
Más posibilidades.*

Virtualización y contenedores



VIRTUALIZACIÓN TRADICIONAL

Varias máquinas en un mismo ordenador físico

Más sistemas operativos,
más posibilidades
en un mismo equipo

¿Qué es la virtualización?

La virtualización permite ejecutar varios sistemas operativos en un mismo ordenador físico, compartiendo sus recursos de hardware.

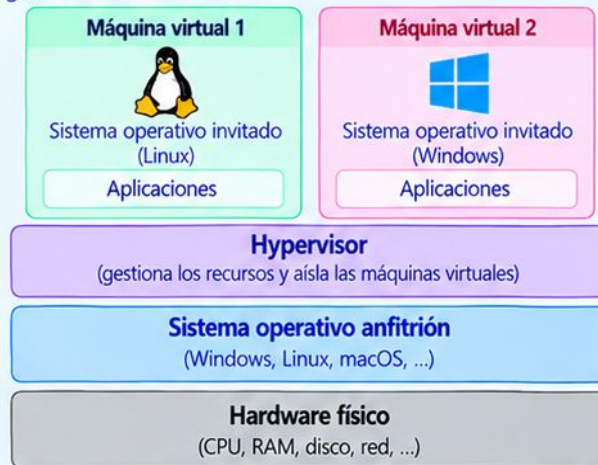


Con la virtualización podemos tener varios ordenadores dentro de un solo ordenador.

Cada máquina virtual se comporta como un ordenador independiente.

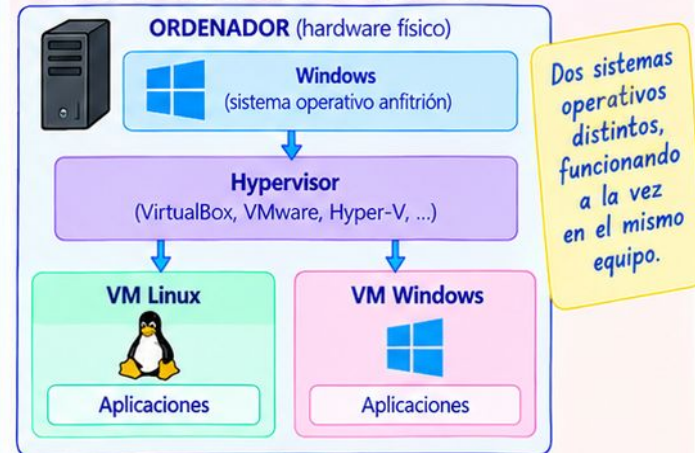
Estructura de la virtualización

El hardware físico se comparte entre varias máquinas virtuales gracias a un software llamado **hypervisor**, que se encarga de gestionar los recursos.



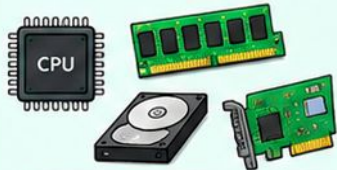
Ejemplo práctico

En un ordenador con Windows podemos usar un hypervisor (como VirtualBox o VMware) para crear varias máquinas virtuales con distintos sistemas operativos.



Hardware físico

Es el ordenador real, con sus componentes: procesador (CPU), memoria RAM, disco duro o SSD, tarjeta de red, etc.



Todos los recursos se comparten entre las máquinas virtuales.

Sistema operativo anfitrión

Es el sistema operativo instalado en el ordenador físico, que permite ejecutar el hypervisor. Ejemplos: Windows, Linux, macOS.



El sistema operativo anfitrión convive con las máquinas virtuales.

Hypervisor

Es el software que crea y gestiona las máquinas virtuales, asignando los recursos de hardware a cada una de ellas y manteniéndolas aisladas.



El hypervisor controla y aísla cada máquina virtual.

Máquinas virtuales

Son ordenadores virtuales que se ejecutan dentro del ordenador físico. Cada una tiene su propio hardware virtual (CPU, RAM, disco, red, ...) y su propio sistema operativo.



Podemos crear tantas como necesitamos, según los recursos disponibles.

Sistemas operativos invitados

En cada máquina virtual se instala un sistema operativo, que funciona de forma independiente del sistema anfitrión y de las demás máquinas virtuales.



Podemos usar diferentes sistemas operativos en cada máquina virtual.



En resumen: la virtualización tradicional permite ejecutar varios sistemas operativos en un mismo hardware mediante un hypervisor, de forma aislada y aprovechando mejor los recursos del equipo.

Un solo ordenador.
Muchas posibilidades.



CONTENEDORES

Aplicaciones aisladas, ligeras y listas para funcionar

Misma máquina,
muchas aplicaciones,
cero problemas.

Un nuevo modelo

A diferencia de la virtualización tradicional, los contenedores comparten el sistema operativo del host y aíslan solo lo necesario para que cada aplicación se ejecute de forma independiente.

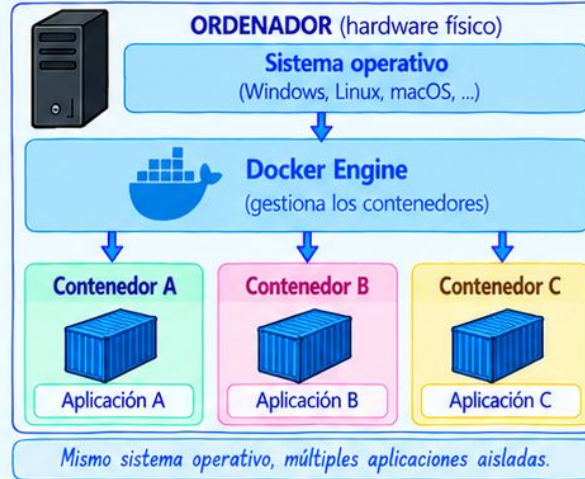
Con contenedores podemos ejecutar varias aplicaciones en el mismo ordenador, sin necesitar varios sistemas operativos.

Son más ligeros, más rápidos y más fáciles de gestionar.



Arquitectura con contenedores

En un solo sistema operativo ejecutamos Docker Engine, que se encarga de crear y gestionar los contenedores.



¿Qué es un contenedor?

Un contenedor es un entorno de ejecución aislado que incluye la aplicación y todo lo necesario para que funcione: dependencias, librerías, configuración, etc.

Comparte el sistema operativo del host, pero está aislado de los demás contenedores.



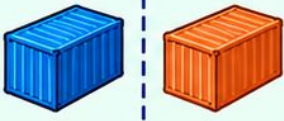
Todo lo necesario para ejecutar tu aplicación, en un contenedor.

Un contenedor es como una caja independiente dentro de tu sistema operativo.

Aspectos clave de los contenedores

Aislamiento

Cada contenedor se ejecuta de forma aislada, con sus propios procesos, sistema de archivos, red y recursos.



Las aplicaciones no se interfieren entre sí.

Procesos

Los contenedores ejecutan sus propios procesos, independientes de los demás contenedores y del sistema anfitrión.

```

$ docker ps
CONTAINER ID   NAMES
5f2a3e1c9d2a   app-web
7a1d8c4b6e3f   app-db
  
```

Cada contenedor tiene su propio conjunto de procesos.

Sistema de archivos

Cada contenedor tiene su propio sistema de archivos, basado en una imagen de solo lectura, con una capa escribible para los cambios.



Los cambios en un contenedor no afectan a los demás.

Red

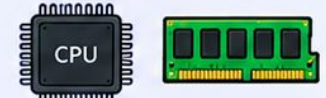
Los contenedores pueden comunicarse entre sí mediante redes virtuales y también acceder a redes externas (por ejemplo, Internet).



Pueden comunicarse de forma segura y controlada.

Recursos

Docker permite limitar el uso de recursos (CPU, memoria, etc.) de cada contenedor, evitando que una aplicación consuma todo el sistema.



Límites de CPU Límites de memoria



¡Una máquina, infinitas posibilidades!

Los contenedores nos permiten ejecutar múltiples aplicaciones de forma aislada, eficiente y controlada, compartiendo el mismo sistema operativo.

Más aplicaciones.
Menos recursos.
Más libertad.



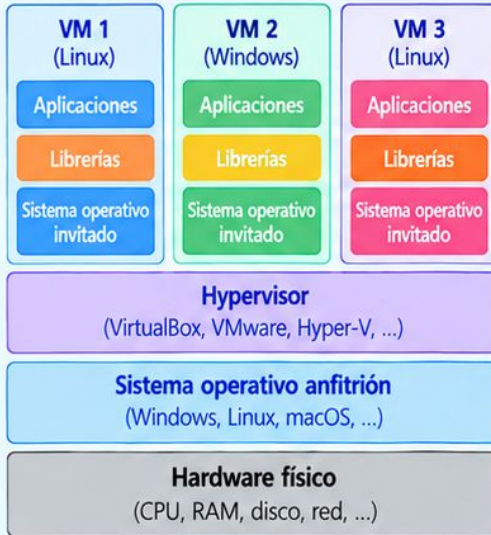
MÁQUINAS VIRTUALES FRENTE A CONTENEDORES

Distintos enfoques, un mismo objetivo: ejecutar aplicaciones

Elige la herramienta adecuada para cada necesidad

Máquinas virtuales (VM)

Varios sistemas operativos en un mismo hardware.



Comparativa

Aspecto	Máquinas virtuales	Contenedores
 Arquitectura	Cada VM incluye su propio sistema operativo invitado.	Comparten el sistema operativo del host. Solo incluyen la aplicación y sus dependencias.
 Consumo de recursos	Alto. Cada VM necesita su propio sistema operativo.	Bajo. Comparten el kernel del sistema operativo.
 Tiempo de arranque	Lento. Puede tardar desde varios segundos hasta minutos.	Muy rápido. Arrancan en milisegundos o pocos segundos.
 Aislamiento	Aislamiento fuerte. Sistemas operativos completamente independientes.	Aislamiento a nivel de procesos. Suficiente para la mayoría de casos, aunque menor que una VM.
 Portabilidad	Menor. Las VM suelen ser más grandes y dependen del hypervisor.	Mayor. Las imágenes son ligeras y se ejecutan en cualquier sistema con Docker.
 Casos de uso	- Diferentes sistemas operativos - Aplicaciones que requieren un SO completo - Entornos muy aislados - Laboratorios y pruebas	- Despliegue de aplicaciones web - Microservicios - Integración y entrega continua - Entornos de desarrollo y pruebas - Escalado rápido

Contenedores

Aplicaciones aisladas, ligeras y portátiles.



Los contenedores comparten el sistema operativo, pero están aislados entre sí.



Las máquinas virtuales son como tener varios ordenadores dentro de un mismo ordenador.

Los contenedores son como cajas independientes, listas para ejecutar aplicaciones de forma rápida y eficiente.

Un contenedor no es una máquina virtual.

Ambos aíslan, pero funcionan de forma diferente y están pensados para necesidades distintas.

¿Cuál elegir?

Depende de tus necesidades: si necesitas ejecutar diferentes sistemas operativos o un fuerte aislamiento, usa máquinas virtuales. Si buscas rapidez, eficiencia y portabilidad para tus aplicaciones, los contenedores son la mejor opción.



En resumen: las máquinas virtuales y los contenedores son herramientas complementarias. Conocer sus diferencias te permitirá elegir la mejor opción en cada proyecto.

Diferentes caminos, más posibilidades.

Linux, el kernel y docker



LINUX, EL KERNEL Y DOCKER

Contenedores gracias a las funcionalidades del kernel Linux

*El mismo kernel,
infinitas posibilidades*

Más eficiencia
Más aplicaciones
Más flexibilidad
Más innovación

¿Por qué aparece Linux?

Docker utiliza funcionalidades del kernel Linux para implementar el aislamiento de los contenedores.

No crea un sistema operativo independiente para cada contenedor, sino que comparte el mismo kernel entre todos ellos.

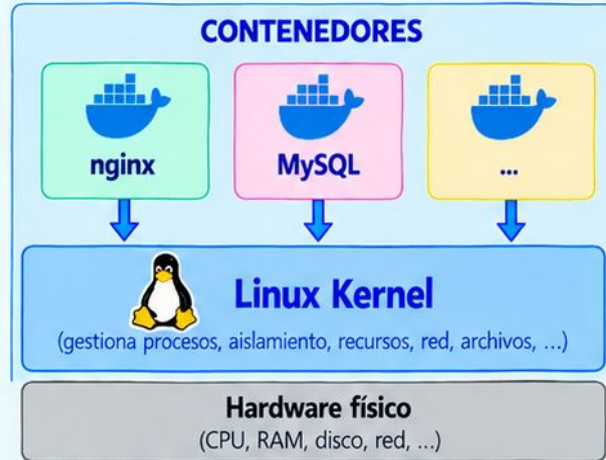


Docker aprovecha las características del kernel Linux para aislar y gestionar los contenedores.

Sin el kernel Linux y sus funcionalidades, los contenedores no serían posibles.

Los contenedores comparten kernel

Todos los contenedores se ejecutan como procesos en el mismo sistema operativo (Linux), compartiendo su kernel, pero aislados entre sí mediante distintas funcionalidades.



Mismo kernel, múltiples contenedores, totalmente aislados.

¿Qué aporta el kernel Linux?

El kernel Linux proporciona mecanismos que permiten que cada contenedor tenga su propio espacio aislado, compartiendo el mismo sistema operativo.



Procesos

Cada contenedor ejecuta sus propios procesos, que son gestionados por el kernel.

Los contenedores son procesos del sistema.



Namespaces

Aíslan los recursos del sistema para que cada contenedor tenga su propio espacio de procesos, red, sistema de archivos, etc.

Cada contenedor ve su propio mundo.



Cgroups

Limitan y controlan el uso de recursos (CPU, memoria, I/O, ...) de cada contenedor.

Evitan que un contenedor consuma todos los recursos.



Sistema de archivos

Cada contenedor tiene su propio sistema de archivos, basado en capas (imágenes de solo lectura + capa de escritura).

Aislados, pero compartiendo el mismo kernel.



Red

Cada contenedor tiene su propia interfaz de red y puede comunicarse con otros contenedores y con redes externas.

Redes aisladas, seguras y flexibles.

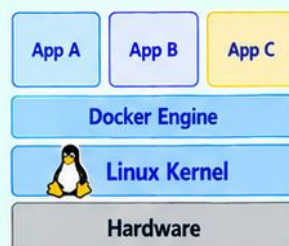
Comparativa con máquinas virtuales

Máquinas virtuales (VM)



Cada VM tiene su propio sistema operativo.

Contenedores (Docker)



Todos los contenedores comparten el mismo kernel.

Ventajas de este enfoque

- ✓ **Más ligeros:** no es necesario un sistema operativo completo por contenedor.
- ✓ **Arranque más rápido:** se inician en segundos.
- ✓ **Menor consumo de recursos:** comparten el kernel.
- ✓ **Mayor densidad:** podemos ejecutar muchos más contenedores en el mismo hardware.
- ✓ **Suficiente aislamiento:** ideal para la mayoría de aplicaciones.

El kernel Linux actúa como un árbitro, asegurando que cada contenedor solo vea y use lo que le corresponde.

En resumen

Docker utiliza las funcionalidades del kernel Linux (procesos, namespaces, cgroups, sistema de archivos y red) para ofrecer contenedores aislados, ligeros y eficientes, todos ellos ejecutándose sobre el mismo sistema operativo.





PRÁCTICA GUIADA: NGINX EN DOCKER

Paso a paso, de lo simple a lo completo

Misma tecnología,
nuevos aprendizajes

Más práctica
Más confianza
Más habilidades
Más proyectos

En esta práctica vamos a crear un contenedor con Nginx y configurarlo paso a paso hasta que sirva el contenido de una carpeta de nuestro sistema.

¡Tú puedes!
Sigue los pasos y comprueba
el resultado en tu navegador.

1. Contenedor básico

Creamos un contenedor a partir de la imagen de Nginx, sin configurar nada más.

```
$ docker run nginx
```



Se descarga la imagen (si no existe) y se ejecuta un contenedor con la configuración por defecto.

Pulsa **Ctrl + C** para detener el contenedor (está en primer plano).

2. Personalizar el puerto

Ejecutamos Nginx en segundo plano y publicamos el puerto 8090 de nuestro equipo.

```
$ docker run -d -p 8090:80 nginx
```



Ahora podemos acceder a Nginx desde nuestro navegador en:

<http://localhost:8090>

La opción **-p 8090:80** publica el puerto 80 del contenedor en el puerto 8090 de tu equipo.

3. Asignar un nombre

Creamos el contenedor con un nombre para gestionarlo más fácilmente.

```
$ docker run -d --name mi-nginx \
-p 8090:80 nginx
```



¡Ahora ya tiene nombre!

Podemos comprobar que está en ejecución con:

```
$ docker ps
```

El nombre nos permite gestionar el contenedor fácilmente (iniciar, detener, eliminar, ...).

4. Servir una carpeta del anfitrión

Por último, hacemos que Nginx sirva el contenido de una carpeta de nuestro sistema anfitrión.

```
$ docker run -d --name mi-nginx \
-p 8090:80 \
-v "$(pwd)/html":/usr/share/nginx/html \
nginx
```



Nginx sirve el contenido de tu carpeta

Ahora, al acceder a <http://localhost:8090> verás los archivos de tu carpeta en lugar de la página por defecto de Nginx.

La opción **-v** monta una carpeta de tu sistema en el directorio donde Nginx espera los archivos web.



¡Paso a paso!
Así vas aprendiendo y
entendiendo cada opción
del comando.



Comandos útiles

<pre>\$ docker ps</pre>	Lista los contenedores en ejecución.
<pre>\$ docker ps -a</pre>	Lista todos los contenedores.
<pre>\$ docker stop mi-nginx</pre>	Detiene el contenedor.
<pre>\$ docker rm mi-nginx</pre>	Elimina el contenedor.
<pre>\$ docker logs mi-nginx</pre>	Muestra los logs del contenedor.



Comprueba el resultado

- ✓ ¿Accede a <http://localhost:8090>?
- ✓ ¿Ves tu página web (index.html)?
- ✓ ¿Puedes modificar los archivos de la carpeta y ver los cambios en el navegador?



Prueba a crear un archivo `index.html` sencillo en tu carpeta con un mensaje de bienvenida. ¡Verás cómo se actualiza al instante!



Docker te da el poder de llevar tus aplicaciones a cualquier entorno.

Pequeños pasos,
grandes proyectos.

Docker en Windows y “docker desktop”



DOCKER EN WINDOWS

Contenedores Linux en tu ordenador Windows

Mismo Windows,
las mismas posibilidades

Más desarrollo
Más práctica
Más entornos
Más futuro

El problema

Docker no puede ejecutar contenedores Linux directamente sobre Windows porque Windows tiene su propio kernel, que es diferente al kernel Linux.

Windows



- Tiene su propio kernel (Windows NT).
- No incluye las funcionalidades del kernel Linux.

Contenedores Linux



- Necesitan funcionalidades del kernel Linux (namespaces, cgroups, ...).
- No pueden ejecutarse directamente sobre el kernel de Windows.

Windows y Linux son sistemas operativos diferentes, con kernels distintos.

La solución: Docker Desktop + WSL 2

Docker Desktop crea y gestiona una máquina Linux ligera utilizando WSL 2 (Windows Subsystem for Linux), donde se ejecuta el Docker Engine y, sobre él, los contenedores Linux.

Docker Desktop te proporciona todo lo necesario para ejecutar contenedores Linux en Windows de forma sencilla.



¿Qué es WSL 2?

- Es un subsistema de Windows que permite ejecutar un kernel Linux real y ligero.
- Integrado en Windows 10/11.
- Ofrece alto rendimiento y mejor compatibilidad que la versión anterior (WSL 1).
- Docker Desktop utiliza WSL 2 como backend habitual.

Tú sigues usando Windows, pero Docker se encarga de la parte Linux.

La máquina Linux de Docker Desktop



WINDOWS

Tu sistema operativo (Windows 10/11)



Docker Desktop

Gestiona WSL 2 y te ofrece una interfaz gráfica



Entorno Linux (WSL 2)

Máquina virtual ligera con kernel Linux



Docker Engine

Ejecuta y gestiona los contenedores



nginx



mysql



tomcat

Múltiples contenedores Linux ejecutándose en tu Windows

Ventajas de esta arquitectura

- ✓ Puedes usar contenedores Linux en tu ordenador Windows.
- ✓ No necesitas instalar un Linux completo.
- ✓ Integración con el sistema de archivos de Windows.
- ✓ Buen rendimiento gracias a WSL 2.
- ✓ Herramientas gráficas incluidas (Docker Desktop).
- ✓ Ideal para desarrollo y aprendizaje.

Todo de forma sencilla, sin configuraciones complejas.

Algunos detalles importantes



- Docker Desktop se encarga de crear, iniciar y gestionar la máquina Linux (WSL 2).
- Puedes acceder a tus archivos de Windows desde los contenedores (por ejemplo, montando volúmenes).
- Aunque los contenedores se ejecutan en Linux, los gestionas desde Windows con la misma Docker CLI (docker run, docker ps, ...).
- Necesitas tener WSL 2 habilitado en tu sistema (la instalación de Docker Desktop lo hace automáticamente).

Tú te centras en desarrollar, Docker Desktop se encarga del resto.

En resumen



- ✓ Windows tiene su propio kernel.
- ✓ Docker Desktop utiliza WSL 2 para proporcionar un entorno Linux.
- ✓ En esa máquina Linux se ejecuta el Docker Engine.
- ✓ Sobre el Docker Engine se ejecutan los contenedores Linux.
- ✓ Todo esto te permite usar contenedores Linux en tu Windows de forma fácil y segura.

¡Mismo Windows, contenedores Linux, sin complicaciones!

*Un solo ordenador.
Dos mundos.
Muchas posibilidades.*





DOCKER DESKTOP

La herramienta que lo hace todo más fácil

Desarrollar, probar
y desplegar...
en tu propio equipo

Más simple
Más rápido
Más compatible
Más posibilidades

¿Qué es Docker Desktop?

Docker Desktop es una aplicación para Windows, macOS y Linux que proporciona una interfaz gráfica y todas las herramientas necesarias para trabajar con contenedores de forma sencilla.



Interfaz gráfica

Permite gestionar imágenes, contenedores, volúmenes, redes, etc. de forma visual y sencilla.



Integración con Docker Engine

Se conecta con el Docker Engine que se ejecuta en segundo plano (en Windows a través de WSL 2).



Docker CLI

Incluye la línea de comandos de Docker para que puedas usar todos los comandos habituales (docker run, docker ps, ...).



Integración con WSL 2 (Windows)

En Windows, utiliza WSL 2 para proporcionar un entorno Linux donde se ejecuta Docker Engine.



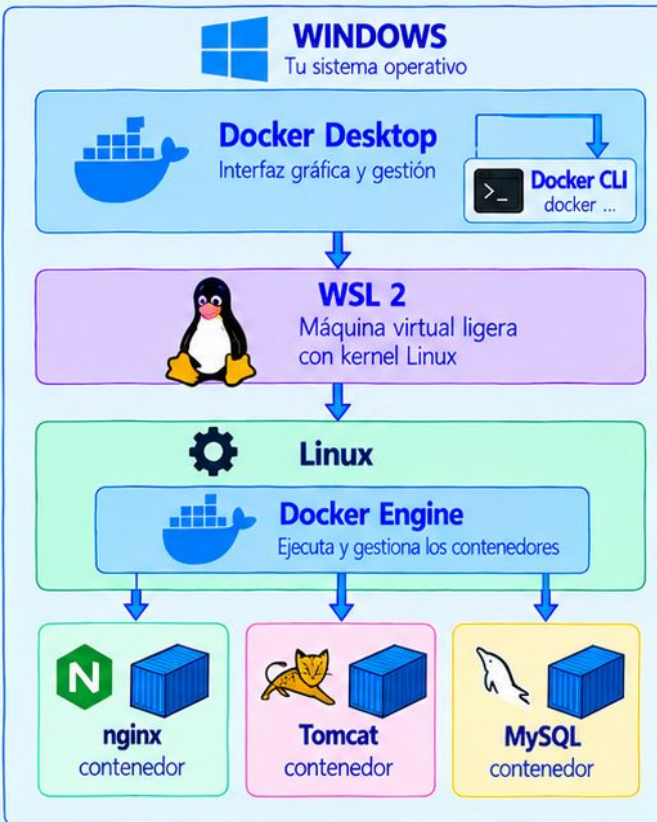
Gestión de recursos

Permite crear, gestionar y monitorizar imágenes, contenedores, volúmenes, redes y configuraciones.



Arquitectura completa en Windows

Docker Desktop se encarga de crear y gestionar toda la infraestructura necesaria para que puedas ejecutar contenedores Linux en Windows, utilizando WSL 2.



Docker Desktop gestiona toda esta infraestructura para ti, sin que tengas que configurar nada manualmente.

¿Qué puedes hacer con Docker Desktop?

- ✓ Descargar imágenes desde Docker Hub.
- ✓ Crear, iniciar, detener y eliminar contenedores.
- ✓ Gestionar volúmenes, redes y configuraciones.
- ✓ Ver logs y estadísticas en tiempo real.
- ✓ Usar la línea de comandos (Docker CLI).
- ✓ Configurar recursos (CPU, memoria, disco).
- ✓ Trabajar con múltiples entornos y proyectos.

Docker Desktop simplifica el trabajo con contenedores, especialmente en Windows, donde configura automáticamente WSL 2 y el entorno Linux necesario.

En resumen



- ✓ Docker Desktop es una herramienta completa para trabajar con contenedores.
- ✓ En Windows utiliza WSL 2 para ejecutar Docker Engine.
- ✓ Incluye interfaz gráfica y Docker CLI.
- ✓ Te permite gestionar imágenes, contenedores, volúmenes y mucho más.
- ✓ Es la forma más sencilla de empezar a trabajar con Docker.



Docker Desktop: la puerta de entrada a un mundo de posibilidades con contenedores.

Mismas ideas.
Más libertad.



PRÁCTICA GUIADA: NGINX CON DOCKER DESKTOP

Crea y configura un servidor web paso a paso, sin usar la línea de comandos

Misma herramienta,
grandes resultados

Más visual
Más sencillo
Más práctico
Más aprendizaje

En esta práctica vamos a crear un contenedor con Nginx utilizando exclusivamente Docker Desktop.

Iremos añadiendo opciones poco a poco: primero el contenedor básico y, al final, lo configuraremos para que sirva una carpeta de nuestro ordenador en el puerto 8090.

¡Paso a paso
hasta tener tu propio
servidor web!

1. Crear un contenedor básico

1. Abre Docker Desktop.
2. En el menú lateral, pulsa **Images** y busca **nginx**.
3. Selecciona la imagen oficial de nginx y pulsa el botón **Run**.



Docker Desktop descargará la imagen (si no la tienes) y creará el contenedor con la configuración por defecto.

2. Asignar un nombre

En la ventana de configuración (Puede aparecer tras pulsar **Run**):

1. En el campo **Name** escribe: **mi-nginx**
2. Mantén el resto de opciones por defecto.
3. Pulsa **Run**.

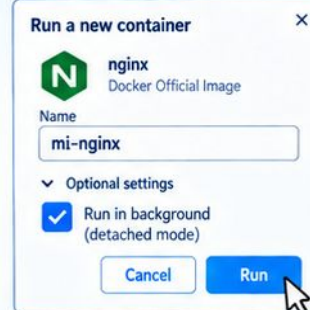


El nombre nos permitirá identificar y gestionar el contenedor más fácilmente.

3. Ejecutar en segundo plano

Para que el contenedor se ejecute en segundo plano:

1. En **Optional settings**, activa la opción **Run in background** (o **-d**).
2. Pulsa **Run**.

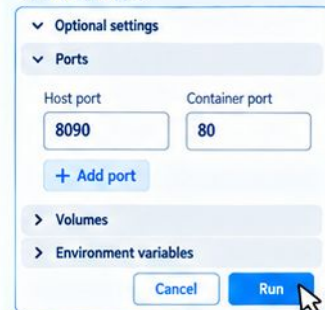


El contenedor quedará ejecutándose en segundo plano y podrás seguir trabajando con Docker Desktop.

4. Publicar el puerto

Vamos a hacer accesible Nginx desde nuestro navegador:

1. En **Optional settings**, pulsa en **Ports**.
2. Añade una regla con:
 - **Host port**: **8090**
 - **Container port**: **80**
3. Pulsa **Run**.



El puerto 8090 de tu equipo se redirecciona al puerto 80 del contenedor.

5. Servir una carpeta del ordenador

Ahora haremos que Nginx sirva el contenido de una carpeta de tu equipo:

1. En **Optional settings**, pulsa en **Volumes**.
2. Selecciona **Bind mount**.
3. Elige una carpeta de tu ordenador (por ejemplo: **C:\web\nginx**).
4. Indica la ruta del contenedor: **/usr/share/nginx/html**
5. Pulsa **Run**.



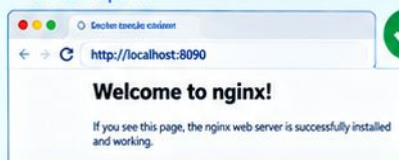
Nginx servirá los archivos de la carpeta de tu ordenador. Si modificas un archivo, se verá el cambio al instante.

6. Comprobar el resultado

1. En el menú **Containers**, localiza tu contenedor **mi-nginx**. Debe aparecer con estado **Running**.
2. Abre tu navegador y accede a:

<http://localhost:8090>

3. Deberías ver la página de bienvenida de Nginx o el contenido de tu carpeta.



¡Enhorabuena!
Tu servidor web
ya está funcionando.

7. Gestionar el contenedor

Desde Docker Desktop puedes realizar todas las acciones:



Puedes detener, iniciar, reiniciar, ver los logs o eliminar el contenedor con un solo clic.

Consejos y siguientes pasos

- ✓ Experimenta modificando los archivos de tu carpeta y recarga la página en sil navegador.
- ✓ Prueba a crear otros contenedores (por ejemplo, Tomcat o MySQL) desde Docker Desktop.
- ✓ Explora las pestañas de configuración (variables de entorno, redes, etc.).
- ✓ Cuando te sientas cómodo, realiza la misma práctica usando la línea de comandos.



¡Ya tienes tu
servidor web con
Docker Desktop!

Ahora puedes hacer
cosas increíbles
con contenedores.



Docker Desktop: la forma más sencilla y visual de trabajar con contenedores.

Crea. Prueba. Despliega.
Sin complicaciones.

Conceptos fundamentales de docker



LOS CONCEPTOS FUNDAMENTALES DE DOCKER

Imágenes, contenedores y su relación

Construye tu entorno, paso a paso

Mismas ideas
Más claridad
Más práctica
Más posibilidades

1. Imagen

Plantilla para crear contenedores

Una imagen es un paquete inmutable que contiene todo lo necesario para ejecutar una aplicación: código, librerías, dependencias y configuración.



Imagen como plantilla

Sirve de modelo para crear uno o varios contenedores.



Inmutabilidad

Una imagen no cambia. Si necesitas una modificación, creas una nueva imagen.



Capas

Se compone de varias capas, reutilizables, lo que hace las imágenes más eficientes.



Imagen base

Muchas imágenes se construyen a partir de una imagen base (por ejemplo, ubuntu, alpine, openjdk, ...).



Tags (etiquetas)

Permiten identificar versiones de la imagen (latest, 1.28, 10, 8, ...).

Ejemplos de imágenes



nginx:latest



tomcat:10



mysql:8



ubuntu:22.04

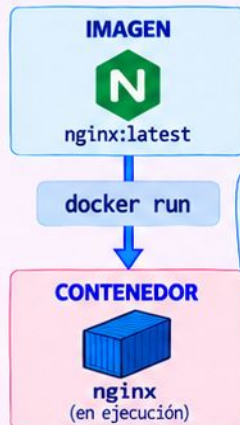


Una imagen es como una receta: siempre igual, y la puedes usar muchas veces.

Una imagen, muchos contenedores. La misma base, infinitas posibilidades.

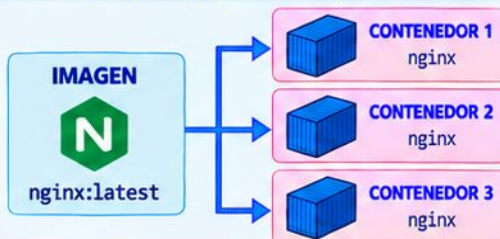
La relación entre imagen y contenedor

A partir de una imagen podemos crear uno o varios contenedores.



El comando `docker run` crea un contenedor a partir de una imagen.

Una imagen puede generar múltiples contenedores



Todos los contenedores creados a partir de la misma imagen son independientes entre sí.

La misma imagen puede dar lugar a muchos contenedores, cada uno con su propia vida.



2. Contenedor

Instancia en ejecución

Un contenedor es una instancia en ejecución de una imagen. Incluye un sistema de archivos, procesos, red y sus propios recursos, aislado del resto de contenedores.



Sistema de archivos

Cada contenedor tiene su propio sistema de archivos, basado en la imagen, y puede incluir cambios temporales.



Procesos

Ejecuta uno o varios procesos (normalmente la aplicación principal y, en algunos casos, servicios auxiliares).



Red

Tiene su propia interfaz de red y puede comunicarse con otros contenedores y con el exterior.



Estado

Por defecto, un contenedor es efímero: cuando se elimina, se pierden los cambios. Para conservar datos se usan volúmenes.



Ciclo de vida

Un contenedor puede crearse, iniciarse, detenerse, pausarse, reanudarse o eliminarse.

Estados de un contenedor



Los contenedores se pueden iniciar, detener, pausar... Tú decides cuándo y cuántos tener en marcha.

Un contenedor es una aplicación en funcionamiento, aislada y lista para usar.



Imágenes y contenedores: piezas clave para construir, ejecutar y desplegar aplicaciones con Docker.

Pequeños bloques, grandes ideas



REGISTRY, REPOSITORIOS E IMÁGENES

Descarga, comparte y reutiliza imágenes

Un mundo de imágenes
para tus proyectos

Más recursos
Más posibilidades
Más colaboración
Más aprendizaje

1. Registry

Un almacén de imágenes

Un registry es un servidor que almacena y distribuye imágenes de contenedores.

Sirve para:

- ✓ Publicar imágenes para que otros puedan usarlas.
- ✓ Descargar imágenes existentes.
- ✓ Gestionar distintas versiones (tags).
- ✓ Compartir imágenes dentro de un equipo u organización.



Docker Hub

Es el registry público más conocido y utilizado. Contiene miles de imágenes oficiales y de la comunidad.

<https://hub.docker.com>

Además de Docker Hub, existen otros registries (públicos y privados), como GitLab Container Registry, Amazon ECR, Azure Container Registry, etc.

4. Descargar una imagen

`docker pull`

Para descargar una imagen desde un registry (utilizando Docker Hub):

```
$ docker pull nginx
```

Este comando descarga la última versión de la imagen `nginx` desde Docker Hub (equivalente a `nginx:latest`).



Con `docker pull` descargas la imagen a tu ordenador. Después podrás crear y ejecutar contenedores a partir de ella.

Solo tienes que descargar la imagen una vez. Luego podrás usar esa imagen todas las veces que quieras.

2. Repository

Colección de imágenes

Un repository (repositorio) es un proyecto o colección de imágenes relacionadas.

En un registry puede haber muchos repositorios, por ejemplo en Docker Hub:

Repositorios (ejemplos)



nginx

Imágenes del servidor web Nginx



tomcat

Imágenes del servidor de aplicaciones Tomcat



mysql

Imágenes de la base de datos MySQL

Cada repositorio puede tener múltiples versiones de la imagen, que se identifican mediante tags.

5. Flujo completo

Del registry al contenedor

Este es el flujo habitual para usar una imagen de Docker Hub:



3. Tags

Versiones de una imagen

Un tag es una etiqueta que identifica una versión concreta de una imagen dentro de un repositorio.

Permite tener varias versiones de la misma imagen.

Ejemplos de tags



`nginx:latest` → Última versión estable
`nginx:1.28` → Versión concreta
`nginx:1.27` → Versión anterior
`nginx:alpine` → Variante más ligera

Si no especificas un tag, se usa por defecto `latest`.

6. Resumen

- ✓ Un registry almacena y distribuye imágenes.
- ✓ Un repository es una colección de imágenes relacionadas.
- ✓ Un tag identifica una versión concreta de una imagen.
- ✓ Con `docker pull` descargas una imagen desde un registry.
- ✓ Con `docker run` creas y ejecutas un contenedor a partir de esa imagen.
- ✓ Docker Hub es el registry público más utilizado.

Millones de imágenes
al alcance de un comando

Una imagen hoy.
Tu aplicación mañana.
El límite lo pones tú.

Descarga,
prueba, experimenta...
¡y construye grandes
proyectos!





EL CICLO DE VIDA DE UN CONTENEDOR

De la imagen al contenedor en ejecución

Misma tecnología,
diferentes estados

Más comprensión
Más claridad
Menos confusión
Más control

1. No son lo mismo

Imagen



- Plantilla inmutable.
- Contiene todo lo necesario para ejecutar una aplicación.
- No se está ejecutando.
- Se almacena en un registry (por ejemplo, Docker Hub).

Ejemplo: nginx:1.28

Contenedor



- Instancia de una imagen.
- Tiene su propio sistema de archivos, procesos y red.
- Puede estar creado, en ejecución o detenido.
- Es temporal: se puede eliminar.

Ejemplo: contenedor nginx

Programa ejecutándose



- Es el proceso principal (y otros) que se ejecuta dentro del contenedor.
- Puede ser un servidor web, una base de datos, etc.
- Se ejecuta cuando el contenedor está en estado "en ejecución".

Ejemplo: nginx (proceso dentro del contenedor)

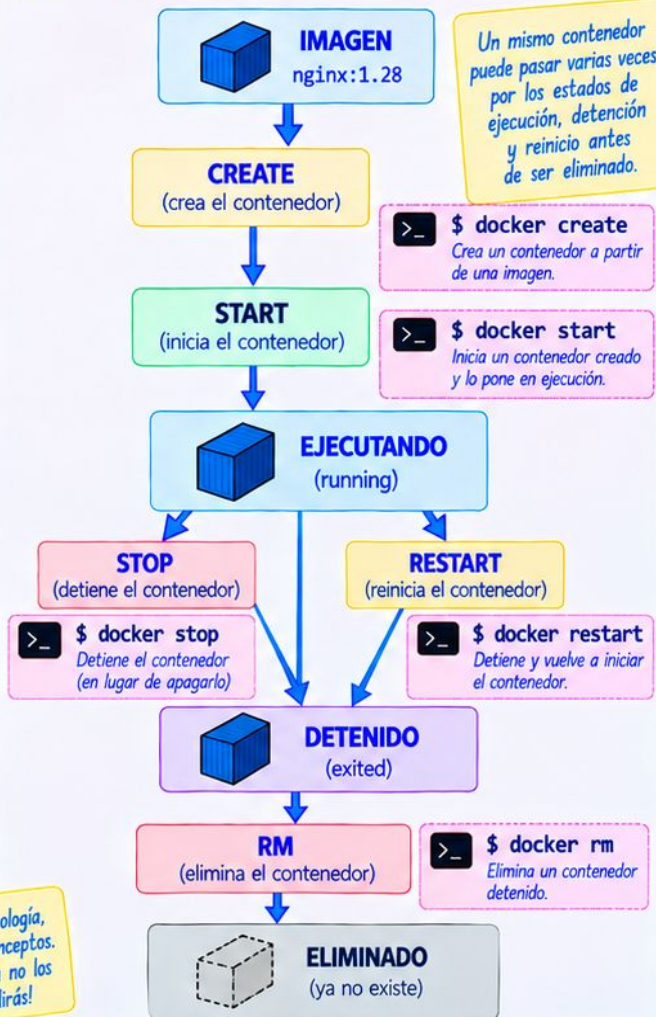


Una imagen es como la receta,
el contenedor es la cocina
y el programa es lo que se está
cocinando dentro.

Misma tecnología,
distintos conceptos.
¡Ahora ya no los
confundirás!

2. Ciclo de vida de un contenedor

Un contenedor pasa por diferentes estados desde que se crea hasta que se elimina.



3. Estados de un contenedor

Un contenedor puede encontrarse en distintos estados:



Creado (created)

- El contenedor existe, pero no se ejecuta.
- Se ha creado con `docker create`.



En ejecución (running)

- El contenedor está ejecutando su(s) proceso(s).
- Se inicia con `docker start` o `docker run`.



Detenido (exited)

- El contenedor ha dejado de ejecutarse.
- Puede volver a iniciarse con `docker start` o `docker restart`.



Eliminado

- El contenedor ya no existe.
- Se elimina con `docker rm`.

4. Comandos principales

Estos son los comandos básicos relacionados con el ciclo de vida:

Comando	Descripción
<code>docker create</code>	Crea un contenedor a partir de una imagen (no lo inicia).
<code>docker start</code>	Inicia un contenedor creado.
<code>docker stop</code>	Detiene un contenedor en ejecución.
<code>docker restart</code>	Detiene y vuelve a iniciar un contenedor.
<code>docker rm</code>	Elimina un contenedor detenido.

5. En resumen

- ✓ Una imagen es una plantilla, no se ejecuta.
- ✓ Un contenedor es una instancia de una imagen.
- ✓ El programa se ejecuta dentro del contenedor.
- ✓ Un contenedor puede estar creado, en ejecución, detenido o eliminado.
- ✓ Con unos pocos comandos puedes controlar todo su ciclo de vida.

Entender el ciclo de vida te dará control y evitará muchos errores.



Imágenes, contenedores y procesos: cada uno tiene su papel, pero trabajan juntos.

Pequeños comandos,
grandes posibilidades.



PUERTOS: COMUNICAR EL CONTENEDOR CON EL EXTERIOR

Expón tus aplicaciones al mundo

Un puente entre
tu aplicación
y los usuarios

Más servicios
Más accesibilidad
Más práctica
Más posibilidades

1. Puerto del contenedor

Cada aplicación dentro del contenedor escucha en un puerto concreto.



Tomcat (dentro del contenedor)
Escucha en el puerto 8080

puerto 8080

El puerto del contenedor es el puerto donde la aplicación está escuchando dentro del contenedor. Este puerto no es accesible directamente desde tu ordenador.

Un contenedor tiene su propia red interna y sus propios puertos.

2. Publicación de puertos

Para acceder a la aplicación desde el exterior debemos publicar el puerto del contenedor en un puerto de nuestro equipo (host).

```
$ docker run -p 8080:80 nginx
```

-p PUERTO_HOST : PUERTO_CONTENEDOR

Puerto de tu ordenador (Windows) donde se recibirá la conexión.

Puerto del contenedor donde está escuchando la aplicación.

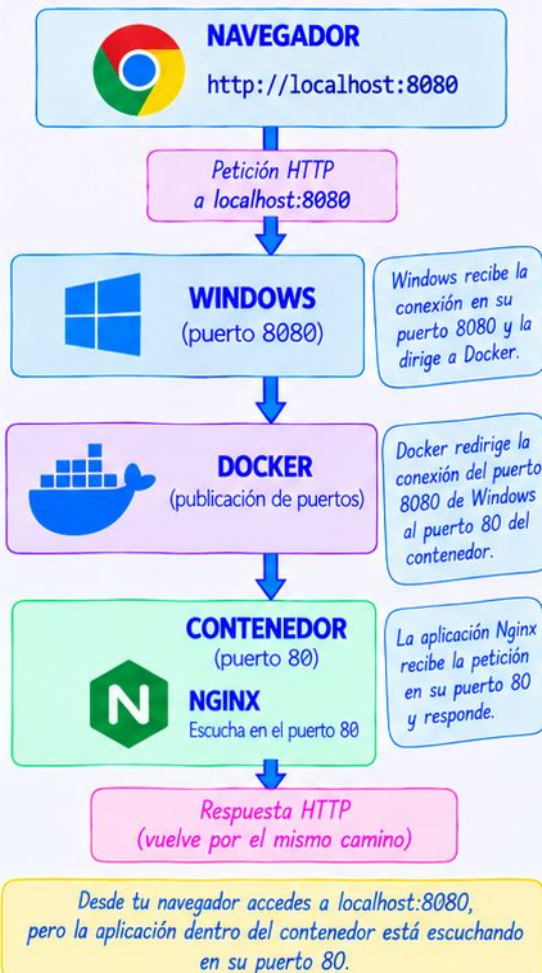
Con **-p** le decimos a Docker que redirija las conexiones del puerto 8080 de nuestro ordenador al puerto 80 del contenedor.

Puedes usar cualquier puerto libre en tu ordenador, aunque lo más habitual es 8080, 8081, etc.



3. Flujo de la conexión

Así es como llega una petición desde tu navegador hasta la aplicación dentro del contenedor.



4. Ejemplos prácticos

Algunos ejemplos muy habituales:

Aplicación	Comando	Acceso desde el navegador
	<code>docker run -p 8080:80 nginx</code>	<code>http://localhost:8080</code>
	<code>docker run -p 8080:8080 tomcat:10</code>	<code>http://localhost:8080</code>
	<code>docker run -p 3306:3306 mysql:8</code>	Se accede con un cliente de base de datos (puerto 3306)
	<code>docker run -p 6379:6379 redis</code>	Se accede con un cliente Redis (puerto 6379)

Puedes cambiar el puerto del host por el que prefieras, siempre que esté libre.

5. Puntos importantes

- ✓ El puerto del contenedor es donde escucha la aplicación.
- ✓ Con **-p** publicamos ese puerto en un puerto de nuestro equipo.
- ✓ Formato: **-p PUERTO_HOST:PUERTO_CONTENEDOR**
- ✓ Podemos tener varios contenedores usando puertos del host diferentes.
- ✓ Si no publicas el puerto, la aplicación solo será accesible desde dentro de la red de Docker.
- ✓ Para aplicaciones web, lo más habitual es usar 8080, 8081, etc.

6. En resumen

- ✓ El contenedor tiene su propio puerto (ej. 80).
- ✓ Con **-p** lo publicamos en un puerto de nuestro equipo. (ej. 8080).
- ✓ Desde el navegador accedemos a `localhost:8080`.
- ✓ Docker se encarga de redirigir la conexión.
- ✓ La aplicación dentro del contenedor recibe la petición y responde.

Mismo concepto, más aplicaciones.
Puertos que conectan ideas.



Los puertos te permiten abrir una ventana a tus contenedores.

Conecta.
Despliega. Aprende.



PERSISTENCIA Y VOLÚMENES

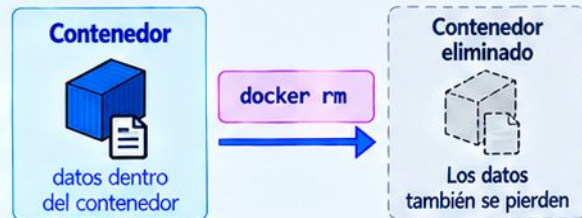
Tus datos a salvo, más allá del contenedor

Los datos también importan

Más seguridad
Más control
Más continuidad
Más posibilidades

1. El carácter efímero del contenedor

¿Qué ocurre con los datos cuando eliminamos el contenedor?



El sistema de archivos de un contenedor es temporal. Cuando se elimina el contenedor, todos los cambios y datos que no estén en un volumen se pierden.

4. Sistema de archivos del contenedor

Un contenedor tiene su propio sistema de archivos, donde se instalan la aplicación, sus configuraciones y se generan datos temporales.



Todo lo que se guarda aquí se perderá cuando se elimine el contenedor, si no se usa un volumen.



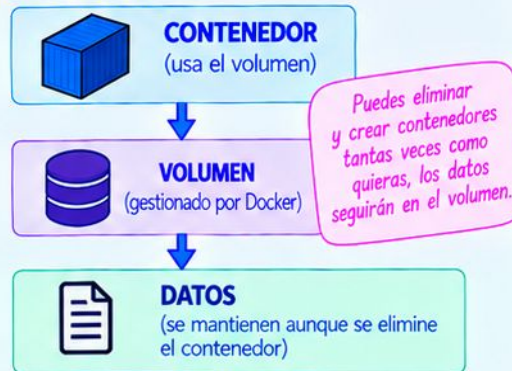
El contenedor es perfecto para ejecutar aplicaciones, pero sus datos no son permanentes.

Si necesitas conservar datos, usa un volumen.

2. Volúmenes

Datos persistentes

Un volumen es un almacenamiento gestionado por Docker para conservar los datos, independientemente del ciclo de vida del contenedor.



5. Volúmenes frente al sistema de archivos del contenedor

Los volúmenes se montan en el contenedor y se utilizan para almacenar los datos que queremos conservar.



3. Comandos de gestión de volúmenes

Algunos comandos útiles para trabajar con volúmenes:

\$ docker volume ls	Lista los volúmenes existentes.
\$ docker volume create datos	Crea un nuevo volumen.
\$ docker volume inspect datos	Muestra información de un volumen.
\$ docker volume rm datos	Elimina un volumen (que no esté en uso).

Los volúmenes son una forma sencilla y segura de conservar datos en Docker.

6. Bind mounts

Usar una carpeta del host

Además de los volúmenes, también podemos montar una carpeta de nuestro sistema operativo en el contenedor (bind mount).



Los bind mounts permiten compartir una carpeta de tu ordenador con el contenedor. Son muy útiles, por ejemplo, para desarrollo, ya que los cambios se ven directamente en el contenedor.

Un bind mount no es un volumen de Docker, es una carpeta de tu sistema de archivos.



Volúmenes para datos persistentes, bind mounts para desarrollo. ¡Tú decides!

Lo importante es entender qué datos quieres conservar y elegir la mejor opción.



Persistencia y volúmenes: tus datos, siempre disponibles.

Contenedores hoy, datos mañana.

Docker CLI (Command Line Interface)



DOCKER CLI: COMANDOS MÍNIMOS

Lo esencial para empezar a trabajar con Docker

Pocos comandos,
grandes posibilidades

Más práctica
Más confianza
Más proyectos
Más aprendizaje

¿Qué es Docker CLI?

Docker CLI (Command Line Interface) es la línea de comandos de Docker. Te permite gestionar imágenes, contenedores, volúmenes y mucho más desde una terminal.

```
$ docker --version
```

```
Docker version 27.1.1, build a1b2c3d
```

Con unos pocos comandos puedes hacer mucho. Aquí tienes una referencia rápida de los comandos más importantes.



Consejos

- ✓ Puedes ver la ayuda de cualquier comando con `--help`.
- ✓ Los nombres de imágenes, contenedores y volúmenes pueden ser lo que quieras.
- ✓ En muchos comandos puedes usar solo una parte del identificador.
- ✓ La terminal te permite automatizar tareas con scripts.
- ✓ Practica estos comandos y ganarás confianza rápidamente.

La práctica hace que los comandos sean parte de tu día a día.

Tabla de referencia rápida



Sistema

Información sobre Docker y el sistema

Comando	Descripción
<code>\$ docker version</code>	Muestra la versión de Docker (cliente y servidor).
<code>\$ docker info</code>	Muestra información detallada del sistema y de la configuración de Docker.



Contenedores

Crea y gestiona contenedores

Comando	Descripción
<code>\$ docker run <imagen></code>	Crea y ejecuta un contenedor a partir de una imagen.
<code>\$ docker ps</code>	Lista los contenedores en ejecución.
<code>\$ docker ps -a</code>	Lista todos los contenedores (en ejecución y detenidos).
<code>\$ docker start <contenedor></code>	Inicia un contenedor detenido.
<code>\$ docker stop <contenedor></code>	Detiene un contenedor en ejecución.
<code>\$ docker restart <contenedor></code>	Reinicia un contenedor.
<code>\$ docker rm <contenedor></code>	Elimina un contenedor.



Volúmenes

Gestiona volúmenes para persistencia de datos

Comando	Descripción
<code>\$ docker volume ls</code>	Lista los volúmenes existentes.
<code>\$ docker volume create <volumen></code>	Crea un nuevo volumen.
<code>\$ docker volume inspect <volumen></code>	Muestra información detallada de un volumen.
<code>\$ docker volume rm <volumen></code>	Elimina un volumen.



Imágenes

Gestiona las imágenes de tu equipo

Comando	Descripción
<code>\$ docker images</code>	Lista las imágenes disponibles en tu equipo.
<code>\$ docker pull <imagen></code>	Descarga una imagen desde un registry (por ejemplo, Docker Hub).
<code>\$ docker rmi <imagen></code>	Elimina una imagen de tu equipo.



Inspección

Obtén información y visualiza logs

Comando	Descripción
<code>\$ docker logs <contenedor></code>	Muestra los logs (salida) de un contenedor.
<code>\$ docker inspect <contenedor></code>	Muestra información detallada de un contenedor (en formato JSON).



Terminal

Accede al interior de un contenedor

Comando	Descripción
<code>\$ docker exec -it <contenedor> bash</code>	Abre una terminal Bash en el contenedor (si está disponible).
<code>\$ docker exec -it <contenedor> sh</code>	Abre una terminal sh en el contenedor (alternativa más común).



Con estos comandos tienes todo lo necesario para empezar a trabajar con Docker.

Aprende los comandos, practícalos, combínalos... ¡y construye cosas increíbles!



Docker CLI: aprende hoy los comandos básicos, construye mañana grandes proyectos.

Misma tecnología.
Más futuro.



docker run: el comando que reúne todos los conceptos

De lo simple a lo completo, paso a paso

Misma herramienta,
grandes posibilidades

Más práctica
Más comprensión
Más confianza
Más proyectos

El comando **docker run** nos permite crear y ejecutar un contenedor a partir de una imagen, y en él podemos aplicar muchas opciones: nombre, segundo plano, publicación de puertos, volúmenes y más. Vamos a verlo de forma progresiva.

El mismo comando,
¡cada vez más potente!

Nivel 1

Lo más básico

Ejecuta un contenedor a partir de una imagen.

```
$ docker run nginx
```



Se descarga la imagen (si no existe) y se ejecuta un contenedor.

El contenedor se ejecuta en primer plano, mostrando sus logs en la terminal.

Nivel 2

Asignar un nombre

Podemos asignar un nombre al contenedor para identificarlo fácilmente.

```
$ docker run --name mi-nginx nginx
```



mi-nginx

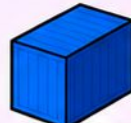
El nombre nos permite gestionar el contenedor más fácilmente.

Nivel 3

Segundo plano

Con la opción `-d` lo ejecutamos en segundo plano (modo desacoplado).

```
$ docker run -d --name mi-nginx nginx
```



mi-nginx

El contenedor se ejecuta detached (en segundo plano), liberando la terminal.

Se ejecuta en segundo plano

Nivel 4

Publicar un puerto

Con `-p` publicamos un puerto del contenedor para acceder desde el exterior.

```
$ docker run -d \
  --name mi-nginx \
  -p 8080:80 \
  nginx
```

localhost:8080



puerto 80
(en el contenedor)

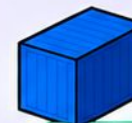
El puerto 8080 de tu equipo se redirige al puerto 80 del contenedor.

Nivel 5

Añadir un volumen

Finalmente, añadimos un volumen para persistir los datos.

```
$ docker run -d \
  --name mi-nginx \
  -p 8080:80 \
  -v datos-nginx:/usr/share/nginx/html \
  nginx
```



mi-nginx



datos-nginx
(volumen)

Los datos del directorio `/usr/share/nginx/html` del contenedor se almacenan en un volumen llamado `datos-nginx`.



Con unas pocas opciones podemos configurar un contenedor listo para usar. ¡Y aún hay muchas más posibilidades!



Conceptos que hemos aplicado

- ✓ Imagen (nginx)
- ✓ Contenedor (docker run)
- ✓ Nombre (--name)
- ✓ Ejecución en segundo plano (-d)
- ✓ Publicación de puertos (-p)
- ✓ Volúmenes (-v)
- ✓ Todos juntos en un solo comando

Cada opción añade una nueva funcionalidad, pero siempre es el mismo comando.

¡De un comando simple a un entorno completo! Así es como se aprende Docker de verdad.

Practica, experimenta y adapta los comandos a tus necesidades. ¡Tú puedes!



docker run: un solo comando, infinitas posibilidades.

Pequeños comandos,
grandes proyectos.